ECES — Junior AI/Data Engineer

Technical Take-Home: Egyptian Housing Market Dataset

> **Time budget:** ~6 hours of real work. You have 5 days to submit.
> **Submit:** a GitHub repo link, plus a walkthrough (written README.md) of your pipeline running end to end.

1. The aim

Egypt has no clean, queryable dataset of what property costs, where, and on what terms. Listing portals hold the information, but the parts that matter most to a researcher — the payment plan, the finishing level, the delivery date, and the compound which it sits in — are not fields. They are buried in free-text descriptions written by hundreds of different agents, in Arabic, in English, and frequently in both at once.

Your job is to turn that free text into a research-grade dataset.

Concretely: produce a dataset that would let an economist answer a question like *"what is the median price per square meter for a 3-bedroom apartment in New Cairo, and how does the cash price compare to the total cost under a 7-year installment plan?"* — without opening a single listing.

The scraping is the easy half. The hard half is extraction a researcher can trust.

2. Input

Source: `https://www.bayut.eg` (Bayut Egypt)

Scope: at least **500 residential listings**, covering both *for sale* and *for rent*, drawn from at least three governorates. Beyond that, the slice is your choice — pick one and justify it in the README.

That is the entire specification of the input. How you get the data out of the site is up to you.

3. Output

3.1 The dataset

One record per listing. Fields split into two groups.

Group A — stated on the page. These exist as structured elements. Getting them right is table stakes.

Field	Notes
`listing_id`	stable across re-runs
`url`	
`purpose`	sale / rent
`property_type`	apartment, villa, chalet, townhouse, duplex, penthouse, studio, land, other
`price`	numeric

Field	Notes
price_period	for rentals: monthly / yearly / null
currency	
bedrooms, bathrooms	integers; studio handled sensibly
area_sqm	numeric — note that some listings hide this
location_raw	as displayed
agency_name	
is_verified	if the site marks it
date_listed	
description_raw	full original text, unedited
language	ar / en / mixed

Group B — extracted from the description. This is where the task lives. None of these are fields on the page; all of them matter to the research.

Field	Notes
compound_name	the project or compound, if any
developer_name	
governorate, city, district	normalized hierarchy, not the raw string
finishing_level	core & shell / semi-finished / fully finished / super lux / furnished / unknown
delivery_status	ready / off-plan
delivery_date	year or year-quarter, if off-plan
sale_type	primary (developer) / resale
payment_type	cash / installments / both
down_payment_amount	numeric
down_payment_pct	numeric
installment_years	numeric
installment_amount	numeric
installment_frequency	monthly / quarterly / annual
cash_discount_pct	if a cash discount is advertised
amenities	list — pool, security, garage, clubhouse, garden, sea view, elevator, etc.
floor_number	
garden_area_sqm, roof_area_sqm	where applicable
is_negotiable	

Two derived fields:

- price_per_sqm — computed; null when area is unavailable
- total_installment_cost — down payment + (installment amount × frequency × years); null when the plan is incomplete

Rules that apply to every field

- `null` **is a correct answer.** If the description does not state it, the value is `null`. A plausible-looking invented delivery date is worse than a missing one, and is scored as worse.
- **Numbers are numbers.** All of these become `1500000`: "1,500,000 EGP", "1.5M", and مليون ونصف (a million and a half).
- **Arabic and English listings produce the same normalized values.** "super lux finishing" and تشطيب سوبر لوكس are the same category.

3.2 Deliverables

1. **The dataset itself** — in XLSX format, so we can inspect it without running anything.
2. **A Clean failure log Summary** — bullet points on problems you faced while scraping and how you got over them (actual problems not coding problems Ex. (bot blockers, captcha, Cloudflare, JS dynamic data injection, etc....)).
3. **An evaluation.** Hand-label **25 listings yourself**: open them, read them, write down the correct values for the Group B fields. Score your pipeline against those 25 and report field-level accuracy, and **hallucination rate** — how often your pipeline produced a value where the truth was `null`.

   If your accuracy is 64%, write 64%. We will trust a candidate who reports 64% with evidence far more than one who claims 97% with none.
4. **A short analysis** (one page, `report.md`) whichever finding surprised you, don't report meaningless statistical aggregations or metrics (average, median, etc...) ask questions and answer them using statistical metrices based on the data you collected. This is the point of the whole exercise: showing us the dataset can answer a real question.
5. **README** — see section 4.

3.3 Properties the pipeline must have

These are outcomes, not instructions.

- **Re-running does not re-fetch what you already have** and do not create duplicate records.
- **It resumes.** If it stops at listing 300, running again continues from 300.
- **You know what it cost.** If you used a paid model, report total tokens and USD (hint: this should be easy if you are using an API provided through openrouter). If you run locally, report wall-clock time and hardware.

4. Methodology

Entirely up to you. Browser automation, direct HTTP, headless framework, hosted LLM, local model, hybrid rules-plus-LLM, regex for the parts where regex genuinely wins — all fine. There is no approach we are looking for and no hidden "right answer."

The one condition: **you must be able to explain and defend every choice** — in the README, and again out loud in the follow-up interview.

Your README should answer:

1. How did you get the data out, and what did you try first that didn't work?
2. Why this extraction method? Where does it fail?
3. What is your `listing_id`, and why does it survive re-runs?
4. If this ran daily for a year unattended, what breaks first?
5. What would you fix with another six hours?

AI coding agents are allowed; we use them daily.

5. How we score it (100 points)

Area	Pts	What earns them
Extraction quality	30	Group B fields correct and normalize; Arabic and English handled equally; payment plans parsed properly
Honest null handling	20	Low hallucination rate — and you measured it
Pipeline robustness	20	Idempotent, resumable, no duplicates, failures logged rather than swallowed
Evaluation & analysis	20	Real gold set, real numbers, and a report.md that says something
Code & explanation	10	Runs from a clean clone; README answers the five questions clearly

Not graded: frontend, UI, test coverage percentage, number of files, clever abstractions. Four well-organized modules is a perfectly good answer.